

杭州电子科技大学

《大数据分析挖掘》

课程大作业报告

作业名称:	Wine Clustering
学 号:	23061919
姓 名:	孙赵炜
指导教师:	黄经州

2025 年 6 月 5 日

实验报告

一 实验要求

本实验围绕 Wine 数据集开展聚类分析，需达成以下要求：

- 运用至少两种聚类方法(如 K-Means、层次聚类等)，对数据集分布特征进行挖掘并实施聚类，对比不同方法的结果差异；
- 针对每种聚类方法，调节关键参数(像 K-Means 的簇数 k 、层次聚类的距离度量方式等)，通过多组实验对比，筛选适配 Wine 数据集的最优参数；
- 涵盖数据加载、预处理、聚类算法实现及结果分析，同时规范撰写实验报告，完整呈现实验流程与结论。

二 实验目的

开展 Wine 数据集聚类实验，旨在以实际数据为依托，深度掌握聚类方法原理与应用。通过对 Wine 数据的分析，理解 K-Means 基于距离划分簇、层次聚类构建层级结构等算法逻辑；借助多方法、多参数的实验对比，挖掘样本在成分、属性维度的分布关联，深化对无监督学习实践价值的认知；同时，在实验过程中，锻炼从聚类指标分析结果、优化方法的能力，为数据分析、机器学习相关实践与研究筑牢基础，提升利用技术解决实际问题的专业素养。

三 实验设计

3.1 实验流程

本次实验首先对给定的数据集进行了数据预处理：将重要的数据列提取出来，并且对数据归一化处理，使得各个数据集满足独立同分布的特性；其次需要确定最优的簇的数量，实验使用肘部法对数据进行分析，得出了最优簇的数量；此后，实验使用 KMeans 聚类和层次聚类两种不同的方法对数据聚类，并且均做出了可视化分析以及比较算法之间的区别；最后实验进行了灵敏度分析，用于探究参数变化对实验算法的影响。具体流程图如下所示：

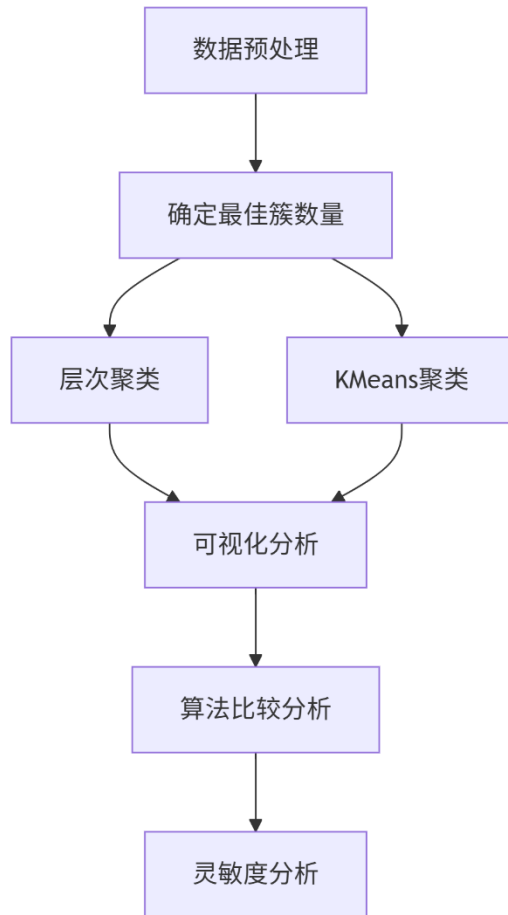


图 1. 实验流程图

3.2 数据预处理

在实验所给的原始数据集中，可能存在着非数字类型的数据项，因此首先需要对数据进行统一处理，将数据的类型全部转换为数字类型。其次在聚类分析中，不同类别的单位并不相同，需要对每个特征进行无量纲化即对数据进行归一化处理，具体公式如下：

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

其中 μ 为该特征的期望， σ 是标准差。通过标准化数据集可以使得每个特征均满足高斯分布，可以是数据集消除量级给分析带来的影响。

3.3 确定簇数量

聚类分析不同于本学期的其他算法，其属于无监督的机器学习，因此在进行聚类的初期需要首先确定簇的数量。聚类的原理是通过设置 k 个质心通过不断的迭代分析出最终的类别，因此实验首先需要确定簇的数量而本次实验主要使用了

肘部法和轮廓系数两种方法确定最佳簇的数量。

3.3.1 肘部法

肘部法是一种用于确定聚类算法中最优聚类数量的启发式方法，其核心思想是：随着聚类数量 k 的增加，样本到其所属聚类中心的距离总和惯性或簇内平方和会逐渐减小。当 k 小于最佳数量是，增加 k 会显著降低损失函数；当 k 达到最佳数量时，损失函数只会微小下降。此时曲线会形成明显的拐点，该点对应的 k 值为最佳簇数量。其中损失函数的具体公式如下：

$$l_{(k)} = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 \quad (2)$$

其中平方项为样本到聚类中心的欧氏距离的平方，因此损失函数代表了所有样本到其所述聚类中心的平方和。肘部法的具体代码如下所示：

```
wcss = []
silhouette_scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
    if k > 1:
        labels = kmeans.labels_
        score = silhouette_score(X_scaled, labels)
        silhouette_scores.append(score)
    else:
        silhouette_scores.append(0)
```

肘部法首先选择了一系列的可能 k 值，并对每个值执行聚类算法，计算损失函数，从而绘制出损失函数随 k 变化的曲线，最终可以找到曲线的拐点。代码的运行结果如下所示：

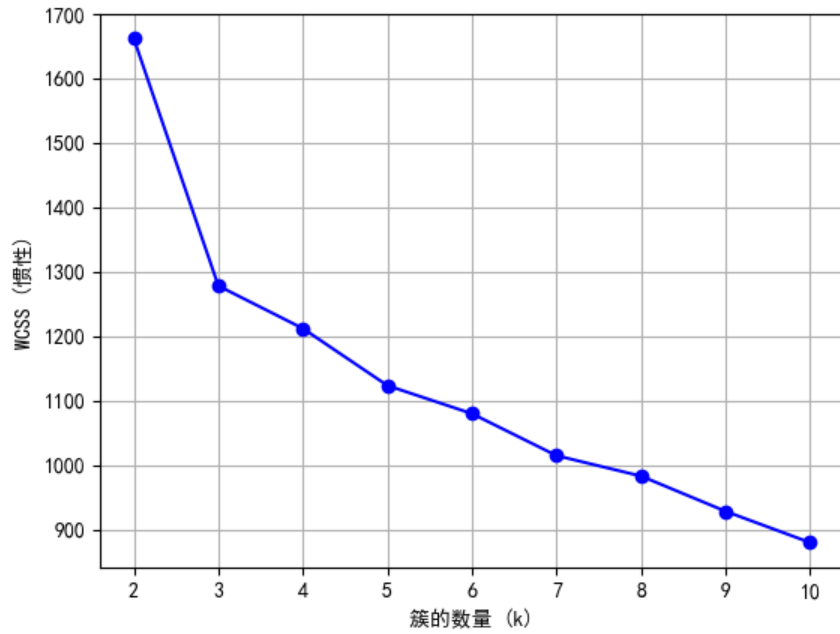


图 2. 肘部法分析最佳簇的数量

从肘部图中可以看出，当 k 的取值为 3 时，损失函数的变化量最大，因此可以将簇的数量定义为 3。

3.3.2 轮廓系数

使用肘部法可能会因为个人主观因素不同导致判断的拐点位置不同，同时肘部法的计算成本较高，需要多次运行聚类算法，因此本实验引入了轮廓系数的方法选择具体最优的簇的数量。

轮廓系数是一种评估聚类质量的指标，用于衡量样本与其所在聚类的紧密程度以及与其他聚类的分离程度。它结合了内聚度和分离度两个维度。轮廓系数的取值越大表示聚类的效果越好。其具体的公式可表示为：

$$s_{(i)} = \frac{b_{(i)} - a_{(i)}}{\max\{a_{(i)}, b_{(i)}\}} \quad (3)$$

其中 $a_{(i)}$ 表示为样本 i 到其所在聚类其他样本的平均距离，表示了内聚度，其取值越小反映出样本 i 与其所在聚类的其他样本越紧密； $b_{(i)}$ 表示为样本 i 到最近邻聚类的所有样本的平均距离，表示分离度，其取值越大，说明样本 i 与其最近邻聚类的分离程度越高。

其中内聚度与分离度的具体公式如下：

$$\begin{cases} a_{(i)} = \frac{1}{|C_i|} \sum_{j \in C_i, j \neq i} dist(i, j) \\ b_{(i)} = \min_{k \neq i} (\frac{1}{|C_k|} \sum_{j \in C_k} dist(i, j)) \end{cases} \quad (4)$$

当内聚度大于分离度时，表示聚类良好；小于分离度时，表示错误分类到了其他聚类中；相等时，表示位于两个聚类的边界上。

轮廓系数的具体代码如下所示：

```
if k > 1:
    labels = kmeans.labels_
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)
else:
    silhouette_scores.append(0)
```

通过代码运行可以得出结果如下所示：

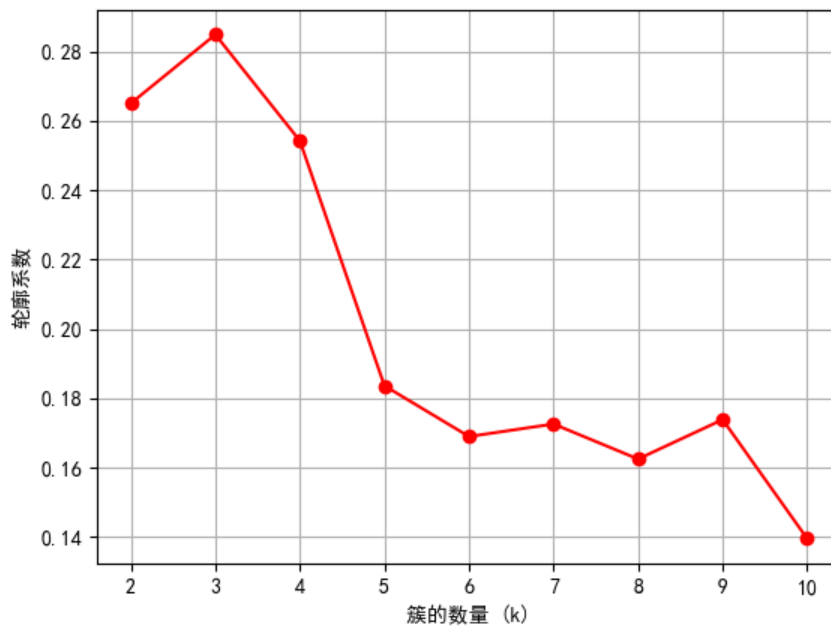


图 3. 轮廓系数确定簇的数量

从图中可以得出最佳的簇的数量为 3，与肘部法得出的结论相同。

3.4 Kmeans 聚类

KMeans 是一种经典的无监督聚类算法，通过迭代优化将数据集划分为 K 个互不重叠的簇，其核心思想是最小化簇内样本与簇中心的距离平方和。其核心目

标是将样本划分到簇中，使得每个簇内样本到其簇中心的距离平方和最小，因此其目标函数可定义为：

$$J = \sum_{i=1}^n ||x_i - \mu_i||^2 \quad (5)$$

其中距离选取了欧氏距离， μ 为簇的中西， c 为样本所属的簇。

Kmeans 的算法伪代码如下所示：

输入：样本 x_i ，簇的数量 k

过程：

1. 随机选取 k 个样本作为簇的中心；
2. 对每个样本，计算其到所有簇的中心的距离，分配到最近的簇中；
3. 对每个簇，更新簇中心为簇所有样本的均值；
4. 重复前两个步骤，知道簇分配不在变化或目标函数收敛；

输出：簇的划分

根据伪代码可以编写出实验的代码，具体代码如下所示：

```
k = 3
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

silhouette = silhouette_score(X_scaled, kmeans_labels)
print(f'K={k} 的轮廓系数: {silhouette:.3f}')

cluster_sizes = np.bincount(kmeans_labels)
for i, size in enumerate(cluster_sizes):
    print(f'聚类 {i} 大小: {size}')

df['kmeans_cluster'] = kmeans_labels
```

在聚类代码的同时 还加入了可视化处理，将数据集以散点图的形式绘制在二维平面内，并且分析了各个特征之间的相关性。

3.5 层次聚类

层次聚类是一种通过构建数据点的层次化聚类树来实现聚类的方法，主要分为自上而下的凝聚式聚类和自顶而下的分裂式聚类，但是其核心思想均为通过计算簇间距离，迭代合并或分裂簇，形成层次化的聚类结构。本次实验选择了使用凝聚式聚类方法。

在层次聚类中，最主要的计算公式为簇间距离度量，用于在对不同簇进行聚类时的主要依据。常见的度量方式有：Ward 连接、全连接以及单链接。实验选

择了使用 Ward 连接的方法，具体公式如下所示：

$$d(C_a, C_b) = \sum_{x \in C_a \cup C_b} d(x, \mu_{ab})^2 - (\sum_{x \in C_a} d(x, \mu_a)^2 + \sum_{x \in C_b} d(x, \mu_b)^2) \quad (6)$$

Ward 方法是一种基于方差最小化的策略，适合发现球形簇并抵抗噪声干扰，并且与 Kmeans 的目标函数一致，有较强的鲁棒性。

层次聚类的伪代码如下所示：

输入：样本集 D；簇数 k；

过程：

1. 将每个数据点设置为独立的簇，计算所有点之间的距离；
2. 找到距离最小的两个簇，合并为新的簇；
3. 计算新簇与其他簇的距离；
4. 重复前两个步骤，直到簇数量达到预设值；

输出：簇的划分

根据伪代码可以编写实验的代码，具体代码如下所示：

```
n_clusters = 3
hierarchical = AgglomerativeClustering(n_clusters=n_clusters, linkage='ward')
hierarchical_labels = hierarchical.fit_predict(X_scaled)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

silhouette_hierarchical = silhouette_score(X_scaled, hierarchical_labels)
print(f'层次聚类轮廓系数: {silhouette_hierarchical:.3f}')

cluster_sizes = np.bincount(hierarchical_labels)
for i, size in enumerate(cluster_sizes):
    print(f'聚类 {i} 大小: {size}')

df['hierarchical_cluster'] = hierarchical_labels
```

在层次聚类同时还绘制了算法生成的树状图以及其他可视化分析。

四 实验调试与结果分析

4.1 Kmeans 聚类结果分析

实验使用 Kmeans 算法对数据集进行处理分析，得到了每个簇的大小以及各个特征的均值，绘制了散点图、热力图以及雷达图。

在算法的处理下，得到了每个簇的大小，结果如表 1 所示：

表 1. Kmeans 算法下簇的大小

簇类	大小
聚类 0	65
聚类 1	51
聚类 2	62

由此得出了各个簇在二维散点图中的分布，结果如图 4 所示：

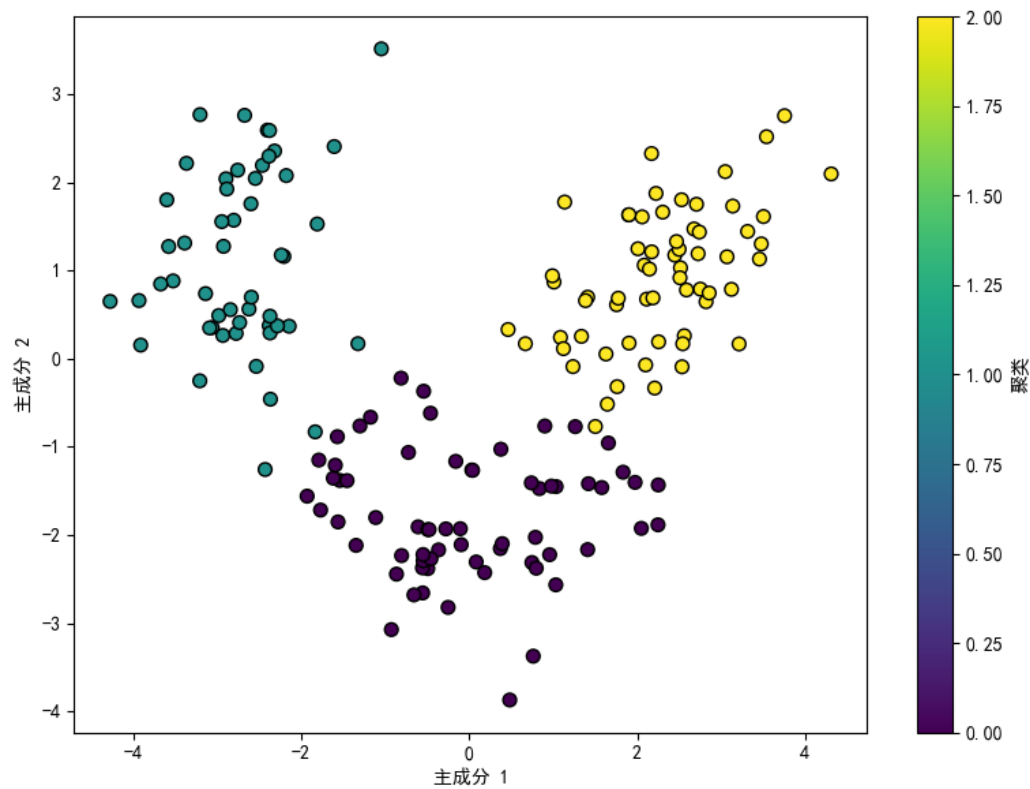


图 4. Kmeans 算法下簇的二维平面分布

本图是在 PCA 降维后绘制的图像，横轴代表了主成分 1，纵轴代表了主成分 2，从中可以看出：三个簇之间有明显的区分线，并且三个簇分别分布于图中的三个角落。

二维的可视化适用于快速展示数据的核心模式、聚类结构和特征相关性，可读性较强但是降维后信息可能会丢失；而三维可视化可以保留更多原始维度信息，接近真实数据分布，呈现更加复杂的空间关系。可视化的运行结果如图 5 所示：

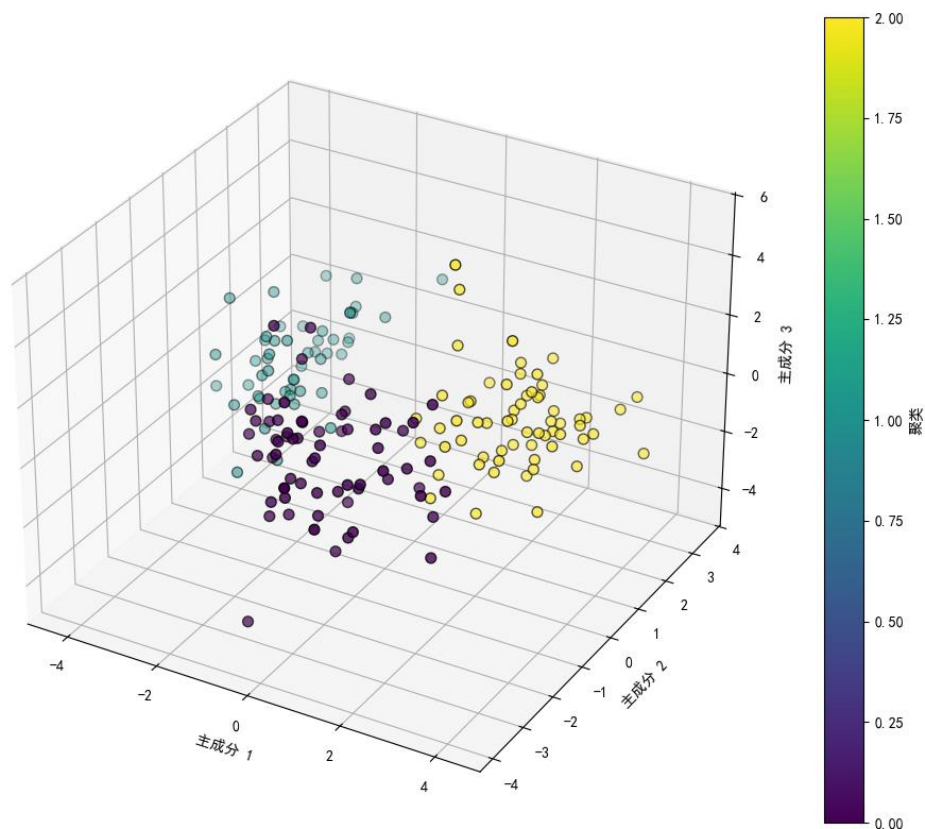


图 5. Kmeans 算法下三维可视化分析

从图中可以看出，在三维的分布下，各个簇之间相互交杂，簇之间的存在灰色地带。由此可见，在主成分为三维的情况下，聚类的效果难以估量。

本实验在调试过程中发现，数据集可以使用非线性降维的方法，专注于保持局部相似性。同时所给数据集的维度较高，考虑到可以使用 t-SNE 算法对数据集进行非线性降维。T-SNE 降维下聚类分析的可视化如图 6 所示：

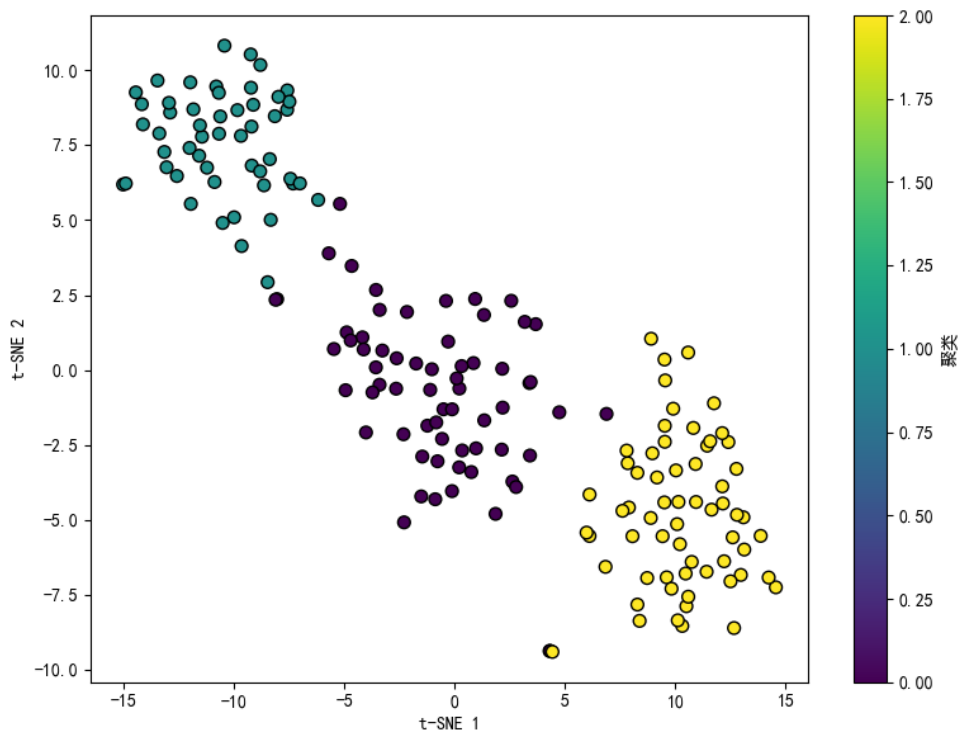


图 6. t-SNE 降维后聚类可视化分析

从图中可以看出簇与簇之间有一定间隔即两个簇之间存在一定的空白区域，说明不同簇在高维空间的特征差异能被 t-SNE 捕捉并在二维呈现出分离，进一步体现了聚类效果有一定合理性。

此外，为了展示不同聚类在各特征上的均值分布，绘制了聚类特征的热力图，结果如图 7 所示：

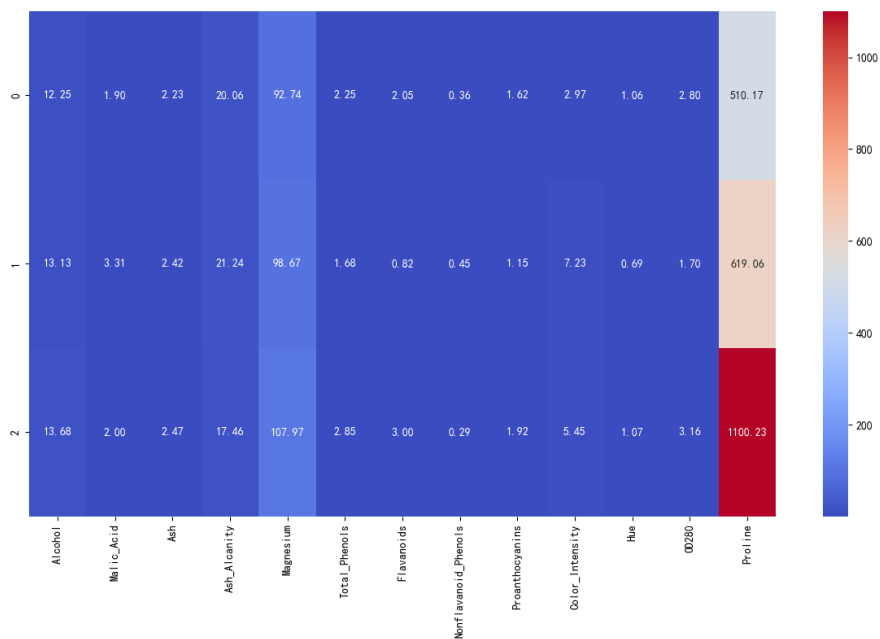


图 7. 聚类特征热力图

在热力图中，颜色越深的特征表明特征均值较高。从图中可得：Proline、Color_Intensity、Flavanoids 的特征差异较大，表明数据集聚类的主要根据为这三种特征。表 2 展示了特征均值最大的三种物质在不同类别上的取值：

表 2. Kmeans 下特征均值最大的物质取值			
簇类	Proline	Color_Intensity	Flavanoids
簇 0	510.17	2.97	2.05
簇 1	619.06	7.23	1.68
簇 2	1100.23	5.45	2.85

在最后计算了 Kmeans 聚类算法下数据集的轮廓系数为 0.285，表明聚类效果较好。

4.2 层次聚类结果分析

实验使用层次聚类法对数据集进行聚类，最终得出了各个簇的大小、数据集在二维平面上的可视化以及层次聚类的树状图。

在代码的运行后，得到各个簇的大小如表 3 所示：

表 3. 层次聚类不同簇的大小	
簇类	大小
簇 0	58
簇 1	56
簇 2	64

同时对层次聚类后的结果进行了可视化分析，使其在二维平面中进行展示，结果如图 8 所示：

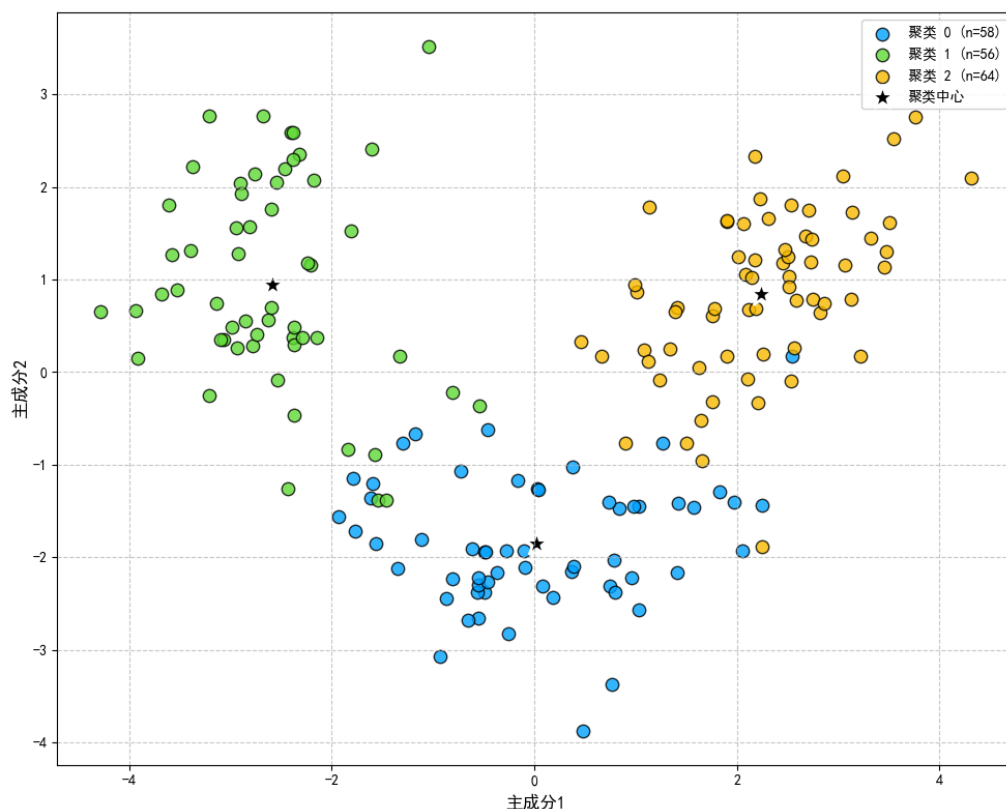


图 8. 层次聚类在二维平面可视化

从图中可得，层次聚类后的簇之间的区分较为模糊，部分点处于其他簇集之间，聚类效果相较 **Kmeans** 聚类较差。

根据层次聚类的算法，可以通过原理绘制层次聚类的树状图，如图 9 所示：

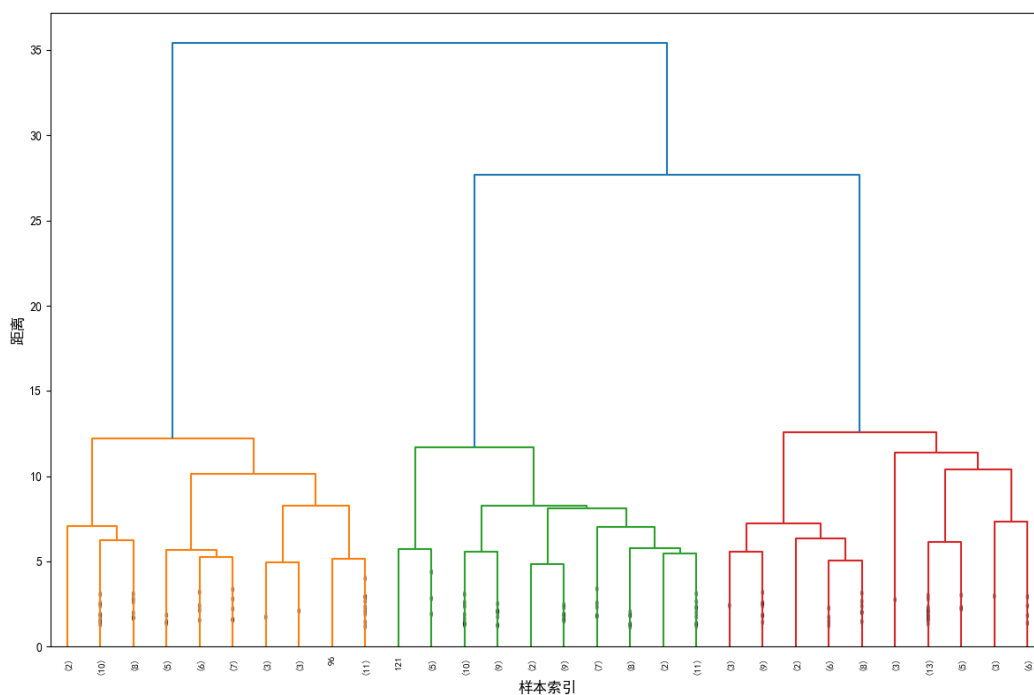


图 9. 层次聚类树状图

根据树状图，可以按照不同的距离得出不同簇的数量，进而可以对簇进行不同的分类。这是层次聚类相较于 Kmeans 算法的优点。

4.3 算法比较分析

通过代码的运算，得出了 Kmeans 聚类和层次聚类的轮廓系数，结果如表 4 所示：

表 4. 两种算法的轮廓系数	
算法	轮廓系数
Kmeans	0.285
层次聚类	0.277

对于聚类算法而言，轮廓系数代表了样本与其所在聚类的紧密程度以及与其他聚类的分离程度，其取值越大表明聚类算法的效果越好。从表中可得，Kmeans 算法的轮廓系数略高于层次聚类，表明 Kmeans 算法的聚类效果较层次聚类更好。

同时观察两种算法在二维平面的可视化图像可得，Kmeans 算法簇之间模糊的区域较少，而层次聚类的模糊区域较多，由此进一步体现出 Kmeans 算法的效果更好。

分析原因可得由于数据集的特点，使得以“簇内紧凑、簇间分离”为目标的 Kmeans 聚类可能会优于通过“凝聚”构建层次化簇结构的层次聚类。数据集中各个指标呈现出连续、近似正态的分布，潜在簇结构更接近球星，更加契合 Kmeans 聚类对凸形簇的假设，同时数据集属于中等规模，Kmeans 可快速收敛得到稳定结果；层次聚类因高复杂度，计算效率低且易陷入距离计算的“细节陷阱”。

4.4 灵敏度分析

在确定最佳簇的数量中，肘部法是根据 Kmeans 算法计算的损失函数，因此需要考虑到肘部法确定的最佳簇数量可能不符合层次聚类，需要对层次聚类的簇的数量进行灵敏度分析，通过对簇的数量进行改变，以观察层次聚类的效果。

实验对层次聚类中的簇的数量在一定区间上变化，通过计算轮廓系数分析最佳簇的数量。在不同簇数量下轮廓系数的值如表 5 所示：

表 5. 层次聚类下不同簇的灵敏度分析	
簇的数量	轮廓系数
2	0.2670
3	0.2774
...	...
9	0.1917
10	0.1986

同时实验还绘制了轮廓系数随簇变化的可视化分析，结果如图 10 所示：

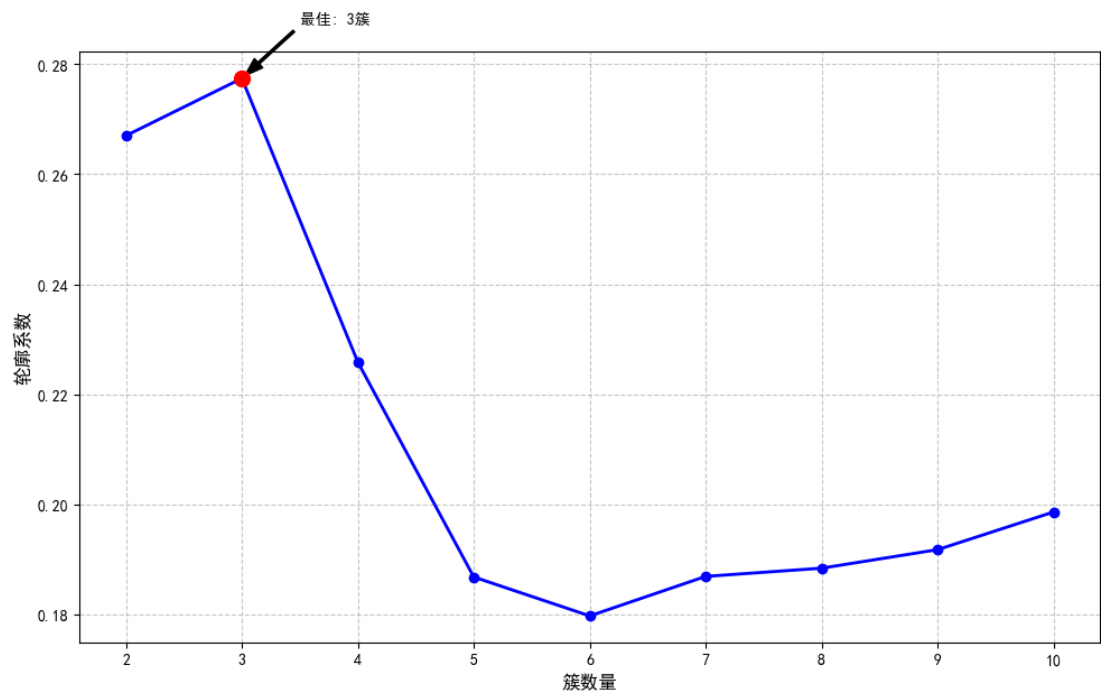


图 10. 轮廓系数随簇变化的可视化

从图中可得，在簇数量为 3 时，层次聚类的效果达到最佳值，因此得出在该数据集中可以使用肘部法确定层次聚类的最佳簇的数量。

4.5 实验完成的任务

本次实验首先对所给数据集进行数据预处理，将数据集中的数据归一化操作，以便后续聚类算法的视线。对于确定簇的数量，使用肘部法和轮廓系数一起比较，确定最佳簇的数量为 3。之后使用 Kmeans 聚类 and 层次聚类两种不同的聚类算法对数据集进行分析，并对结果进行比较，发现 Kmeans 的轮廓系数为 0.285；层次聚类的轮廓系数为 0.277，因此认为 Kmeans 算法聚类效果更佳。对于每个聚类算法，调节了簇的数量这一参数，通过对比选出了最佳簇的数量为 3。最后对两种聚类方法进行了数据可视化分析，使得结果更加直观。